DEVSECOPS CAPSTONE

TaskBoard

From source code to a monitored Kubernetes workload

Application · CI/CD · Image security · Infrastructure as code · Observability

PROJECT WALKTHROUGH

01 **PLAN**
Terraform

02 **BUILD**
GitHub Actions

03 **SCAN**
Trivy gate

04 **SHIP**
GHCR + Helm

05 **OBSERVE**
Prometheus / Grafana

Repository: github.com/ashu273k/DevOps-Final-Project

Technical documentation + submission deck

DOCUMENTATION DECK | OCTOBER 2026

A small app, a complete delivery lifecycle

TaskBoard is the application used to demonstrate the platform—not the end goal by itself.

WEB UI

Frontend experience

React + Vite UI served by Nginx. A task dashboard supports listing, searching and viewing task status.

API

REST API

FastAPI endpoints provide task CRUD operations, health/readiness probes, task statistics and metrics.

DATA

Persistence

PostgreSQL stores application data. SQLAlchemy models and Alembic migrations manage the data layer.

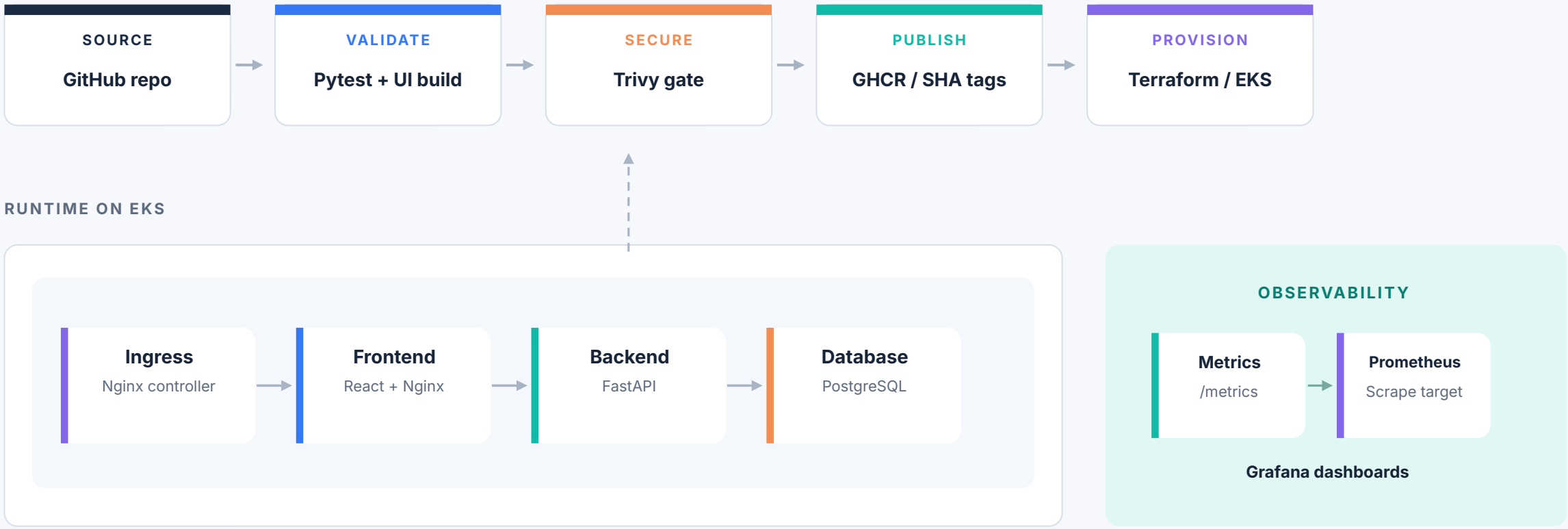
API surface documented by the repository

GET	/api/tasks
GET	/api/tasks/{id}
POST	/api/tasks
PUT	/api/tasks/{id}
DELETE	/api/tasks/{id}
GET	/api/tasks/stats

↗ Application screenshot ↗ OpenAPI docs

End-to-end architecture

Two connected planes: delivery automation and the runtime platform.



Images published by CI are deployed by a separate Helm release in the currently documented workflow.

Technology stack mapped to the repository

APPLICATION

- **React · Vite**
  `frontend/`
- **FastAPI · SQLAlchemy**
  `backend/`
- **PostgreSQL · Alembic**
  `backend/alembic/`

QUALITY + DELIVERY

- **Pytest / SQLite**
  `backend/tests/`
- **Docker / Compose**
  `docker-compose.yml`
- **Actions / GHCR / Trivy**
  `.github/workflows/`

CLOUD PLATFORM

- **Terraform / AWS VPC + EKS**
  `terraform/`
- **Kubernetes / Helm**
  `helm/taskboard/`
- **Bootstrap manifests**
  `k8s/`

OPERATIONS

- **Prometheus values**
  `monitoring/`
- **Metrics + ServiceMonitor**
  `helm/taskboard/`
- **Evidence screenshots**
  `docs/images/`

Design principle Keep the workflow reproducible: code + configuration + versioned images + observable runtime.

Local run and test gates

RUN

One-command stack

docker compose up --build
Frontend: localhost:3000 · API: localhost:8000
PostgreSQL host port: 5433 → container 5432

TEST

Isolated test database

cd backend && pytest -v
Tests use SQLite for the test session, not the Compose PostgreSQL database.

BUILD

Frontend build

cd frontend && npm ci && npm run build
The container uses Node build stage + Nginx runtime.

Useful service endpoints

`/docs`	Interactive OpenAPI	↗ evidence
`/health`	Liveness check	↗ evidence
`/ready`	Database readiness	
`/metrics`	Prometheus format	↗ evidence

Reproducibility note

The screenshots in docs/images are useful evidence. Re-run commands during recording to demonstrate the current environment and avoid relying on screenshots alone.

↗ App running ↗ Pytest output ↗ Compose build

CI/CD: test first, scan before publish

The workflow separates fast validation from publishing. Main-branch image release is gated by tests, frontend build and Trivy.

1 Validate
Pytest + frontend build

→

2 Build images
Frontend + backend Docker

→

3 Scan
Trivy HIGH / CRITICAL

→

4 Publish
GHCR with commit SHA

→

5 Deploy
Separate Helm release

IMAGE TAGGING

```
ghcr.io/<owner>/taskboard-backend:<commit-sha>

ghcr.io/<owner>/taskboard-frontend:<commit-sha>
```

SECURITY GATE

Scan both locally built images. The documented command filters HIGH/CRITICAL severity, ignores unfixed items, and exits non-zero for qualifying findings.

Important: the repository documents CI publication, not an automatic Kubernetes deployment.

↗ Actions success ↗ Push proof ↗ GHCR images

Infrastructure as code: Terraform → AWS EKS

10.20.0.0/16	2 + 2	1	1.31
VPC CIDR	Public + private subnets	NAT gateway	EKS Kubernetes version

Provisioning lifecycle

- `terraform init`
- `terraform fmt -check -recursive`
- `terraform validate`
- `terraform plan`
- `terraform apply`
- `aws eks update-kubeconfig ...`
- `kubectl get nodes`

Current classroom profile

Cluster: taskboard-eks
Region: ap-south-1
AZs: ap-south-1a / ap-south-1b
Node group: t3.micro, one desired node
Network: two public + two private subnets

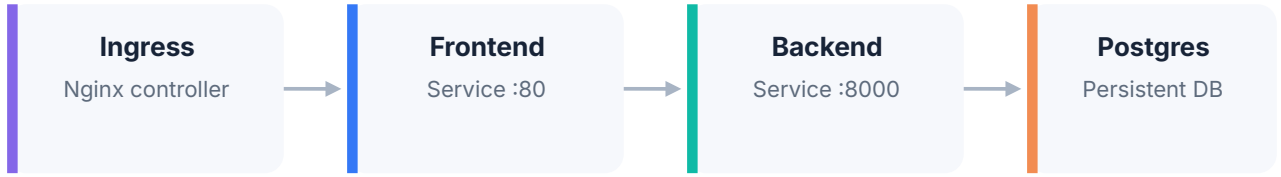
Cost control: AWS control-plane and NAT resources may incur charges. Destroy demo infrastructure after the walkthrough.

↗ Terraform plan ↗ Apply proof ↗ AWS resources ↗ Destroy proof

Kubernetes + Helm: package and operate the workload

Namespace: taskboard · Helm release: taskboard · Ingress hostname: taskboard.local (when configured)

Traffic path



/ → frontend · /api → backend

Helm manages

- Deployments + Services
- Configurable replicas / resources
- Optional Ingress and HPA
- Optional ServiceMonitor

```
helm upgrade --install taskboard ./helm/taskboard \
  --namespace taskboard --create-namespace \
  --values helm/taskboard/values-dev.yaml \
  --set replicaCount=1
```

Rubric-sensitive replica count

Dev values use 1 replica. Rubric target: at least 2 frontend + 2 backend replicas. Verify the chosen values and live pods.

↗ Helm release

↗ Pods + Services

Observability: metrics → Prometheus → Grafana

Instrument FastAPI

prometheus-fastapi-instrumentator
HTTP requests · latency · status codes

→

Scrape with Prometheus

ServiceMonitor selects backend service
/metrics · 15-second interval

→

Visualize in Grafana

Dashboard panels answer operational
questions about traffic and latency

Proof of health

Target = UP
Panel has data

Operational verification commands

```
kubectl get servicemonitor -n taskboard
kubectl get pods -n monitoring
kubectl port-forward -n taskboard \
  svc/taskboard-taskboard-backend 8000:8000
curl http://localhost:8000/metrics
```

Evidence check before upload

The docs index lists metrics and ServiceMonitor screenshots. Explicitly capture Prometheus Targets showing UP and Grafana with a populated panel if the current evidence does not clearly show them.

↗ Metrics output ↗ Metrics panel candidate ↗ ServiceMonitor ↗ Monitoring config evidence

Evidence is already catalogued in docs/images

Use the existing screenshots as supporting artifacts, then confirm critical runtime claims in the live walkthrough.

APPLICATION + TESTS

- ↗ App UI
- ↗ OpenAPI
- ↗ Health
- ↗ Pytest

CI/CD + GHCR

- ↗ Actions run
- ↗ Git push
- ↗ Published images
- ↗ Compose build

TERRAFORM + AWS

- ↗ Terraform plan
- ↗ Apply
- ↗ AWS resources
- ↗ Destroy

KUBERNETES + HELM

- ↗ Helm release
- ↗ Pods / Services

METRICS + MONITORING

- ↗ Metrics
- ↗ Application metric
- ↗ ServiceMonitor
- ↗ Prometheus values/evidence

All links open the corresponding repository file. A screenshot is evidence of a captured state—not a real-time health check.

Submission risks to close before recording

The deck and docs should be accurate about what is configured versus what still needs live proof.

GAP

ArgoCD in the video rubric

The submission instructions explicitly request a synced application / GitOps walkthrough. No ArgoCD configuration is documented in the repo tree/docs inspected. Configure it and show real sync status, or resolve the expectation with the instructor.

CAPACITY

Replica target

The documented development values use one replica. The rubric asks for at least two frontend and two backend replicas. Select values and node capacity that support the target, then verify live pod counts.

EVIDENCE

Prometheus + Grafana screenshots

The evidence index names metrics and ServiceMonitor files, but a dedicated Targets page showing UP and a populated Grafana panel should be checked or captured explicitly.

SCOPE

CI/CD is not Kubernetes auto-deploy

The documented workflow publishes SHA-tagged images to GHCR. Helm is described as a separate release step after EKS is available; do not narrate automatic deployment unless the pipeline actually performs it.

Rule for the final demo Show real output, state limitations plainly, and capture any missing evidence before upload.

Key learnings: reliable delivery is a chain of controls

01	**Reproducibility**	Compose localizes setup; Terraform and Helm turn environment state into reviewable configuration.
02	**Fast feedback**	Tests and the frontend build run before release artifacts are published.
03	**Traceability + security**	Commit-SHA tags connect an image to source; Trivy enforces a security gate before publication.
04	**Operate what you deploy**	Health probes, metrics, Prometheus targets and Grafana panels help confirm runtime behavior.

The architecture is only complete when each handoff has both a defined mechanism and verifiable evidence.

Final submission checklist

M10 is worth 5 points in the rubric. The video is mandatory even if the README and deck are present.

- ○ **Documentation** — Root README explains stack, architecture, setup, CI/CD, Terraform, Kubernetes and observability.
- ○ **Deck** — Commit both demo/presentation.pptx and demo/presentation.pdf.
- ○ **Video** — Record slides first, then live application, source, CI/CD, Terraform/AWS, ArgoCD, Kubernetes and observability.
- ○ **Live proof** — Prometheus target is UP; Grafana panel is populated; pods and replica counts are healthy.
- ○ **Cloud hygiene** — Use `terraform destroy` after the AWS walkthrough; keep secrets/state out of Git.

Repository github.com/ashu273k/DevOps-Final-Project

See docs/06-evidence.md + docs/GRADING_SUBMISSION_MATRIX.md